

NumPy

Cómputo vectorizado, broadcasting y por qué los loops son lentos

Enrique Bolaños Abner Xocop

23 de julio de 2026

Veritas · Libertas · Jvstitia

Agenda

- 1.** ¿Por qué NumPy?
- 2.** El ndarray
- 3.** Vectorización
- 4.** Indexing y slicing
- 5.** Broadcasting
- 6.** Agregaciones
- 7.** Memoria y medición
- 8.** Ejercicios
- 9.** Cierre

El problema con las listas de Python

Una lista de Python no guarda números: guarda **punteros a objetos**.

- Cada `int` es un objeto completo (~28 bytes, no 8)
- Los elementos están dispersos en memoria
- Cada operación pasa por el intérprete, uno por uno

Un **ndarray** de NumPy es lo contrario:

- Un solo bloque **contiguo** de memoria
- Todos los elementos del **mismo tipo** (dtype)
- Las operaciones corren en C, sobre el bloque completo

NumPy es la base de Pandas, scikit-learn y PyTorch

¿Qué tan grande es la diferencia?

Sumar 10 millones de números:

```
1 import numpy as np
2
3 datos_lista = list(range(10_000_000))
4 datos_array = np.arange(10_000_000)
5
6 # Python puro
7 total = 0
8 for x in datos_lista:
9     total += x           # ~700 ms
10
11 # NumPy
12 datos_array.sum()       # ~5 ms
```

Más de 100x de diferencia, con menos código. Hoy vamos a entender por qué.

Crear arrays

```
1 import numpy as np
2
3 a = np.array([1, 2, 3, 4])
4 a.dtype      # dtype('int64')
5 a.shape      # (4,)
6
7 m = np.array([[1, 2, 3],
8               [4, 5, 6]])
9 m.shape      # (2, 3)
10 m.ndim       # 2
11
12 x = np.array([1.0, 2, 3])
13 x.dtype      # dtype('float64') <- promocion automatica
```

El dtype es una decisión, no un accidente: define memoria y precisión.

Fábricas de arrays

```
1 np.zeros((3, 4))           # matriz 3x4 de ceros
2 np.ones(5)                 # [1., 1., 1., 1., 1.]
3 np.arange(0, 10, 2)        # [0, 2, 4, 6, 8]
4 np.linspace(0, 1, 5)       # [0., 0.25, 0.5, 0.75, 1.]
5
6 rng = np.random.default_rng(seed=42)
7 rng.normal(size=(2, 3))    # normal estandar 2x3
8 rng.integers(1, 7, size=10) # 10 tiros de dado
```

Usamos `default_rng(seed)` y no `np.random.seed()`: reproducibilidad explícita, la vamos a exigir en HDTs y proyectos.

Operar sobre el array completo

```
1 precios = np.array([100.0, 250.0, 80.0, 120.0])
2
3 precios * 1.12          # IVA: [112., 280., 89.6, 134.4]
4 precios - precios.mean() # centrar en la media
5 np.log(precios)         # ufunc: log elemento a elemento
6 precios > 100           # [False, True, False, True]
```

Estas funciones universales (**ufuncs**) reemplazan al ciclo `for`: el ciclo sigue existiendo, pero corre **en C, no en Python**.

¿Por qué el loop de Python es lento?

En cada iteración de un `for` sobre una lista, Python:

1. Interpreta el bytecode de la línea (otra vez)
2. Sigue el puntero al objeto (salto aleatorio en memoria)
3. Verifica su tipo (¿es `int`?, ¿es `float`?)
4. Crea un objeto nuevo para el resultado

NumPy hace la verificación de tipo **una sola vez** y recorre memoria contigua, que es exactamente lo que el CPU y su caché aman.

MÁS ALLÁ DEL CÓDIGO

Regla del curso: si escribiste un `for` sobre números, pregúntate primero si existe la versión vectorizada. Casi siempre existe.

Slicing en 1D y 2D

```
1 a = np.arange(10)           # [0 1 2 3 4 5 6 7 8 9]
2 a[2:7]                      # [2 3 4 5 6]
3 a[::2]                      # [0 2 4 6 8]
4 a[::-1]                    # invertido
5
6 m = np.arange(12).reshape(3, 4)
7 m[0, :]                    # primera fila
8 m[:, 1]                    # segunda columna
9 m[1:, :2]                  # submatriz: filas 1+, cols 0-1
```

Igual que las listas de Python, pero en cualquier dimensión.

Máscaras booleanas: el filtro de los datos

```
1 notas = np.array([45, 78, 92, 60, 55, 88])
2
3 mascara = notas >= 61      # [False, True, True, False, False, True]
4 notas[mascara]             # [78, 92, 88]
5 notas[notas >= 61].mean()  # promedio de aprobados
6
7 # Combinar condiciones: & y |, con parentesis
8 notas[(notas >= 61) & (notas < 90)]  # [78, 88]
```

Así se filtra en todo el ecosistema: Pandas usa exactamente esta idea

Cuidado: las vistas comparten memoria

```
1 a = np.arange(5)      # [0 1 2 3 4]
2 b = a[1:4]            # vista, NO copia
3 b[0] = 99
4 a                     # [0 99 2 3 4]  <- cambio a!
5
6 c = a[1:4].copy()     # copia independiente
```

Fuente clásica de bugs

El slicing devuelve una **vista** del mismo bloque de memoria. Si van a modificar, usen `.copy()`. (La indexación booleana y fancy sí devuelven copia.)

Las reglas del broadcasting

¿Cómo opera NumPy dos arrays de formas distintas? Alinea los shapes **desde la derecha** y compara dimensión por dimensión:

1. Si son iguales: compatible
2. Si una es 1: se **estira** para igualar a la otra
3. Si no: error

A	B	Resultado
(3, 4)	(4,)	(3, 4)
(3, 1)	(1, 4)	(3, 4)
(3, 4)	(3,)	error

El estiramiento es **virtual**: no se copia memoria.

Broadcasting en acción

```
1 ventas = np.array([[10, 20, 30],      # tienda A
2                    [40, 50, 60]])     # tienda B
3
4 ventas * 2                          # escalar: se estira a todo
5
6 precio = np.array([1.5, 2.0, 0.5])   # por producto
7 ventas * precio                      # (2,3) * (3,) -> por columna
8
9 meta = np.array([[25], [45]])        # por tienda
10 ventas - meta                       # (2,3) - (2,1) -> por fila
```

Centrar columnas, normalizar, calcular distancias: todo es broadcasting.

Resumir: agregaciones y el eje

```
1 m = np.array([[10, 20, 30],
2               [40, 50, 60]])
3
4 m.sum()           # 210: todo el array
5 m.sum(axis=0)     # [50, 70, 90]: colapsa filas
6 m.sum(axis=1)     # [60, 150]: colapsa columnas
7
8 m.mean(), m.std(), m.min(), m.max()
9 m.argmax()       # indice del maximo (aplanado)
```

Truco para no perderse: `axis=0` significa *el resultado ya no tiene filas* (se colapsó ese eje).

Memory layout y cómo medir

Un array (3, 4) vive como 12 números seguidos + **strides** (cuántos bytes saltar por eje). Por eso:

- `m.T` es gratis: solo cambia los strides, no copia nada
- `reshape` suele ser gratis; recorrer por filas es más rápido que por columnas

En Jupyter, midan siempre antes de optimizar:

```
%timeit datos_array.sum()  
%timeit sum(datos_lista)
```

Correr, medir, comparar: ese es el hábito. Lo van a usar en la HDT 1.

Ejercicio 1: z-score sin loops

Normalizar un array: restar la media y dividir entre la desviación estándar.

```
1 rng = np.random.default_rng(7)
2 alturas = rng.normal(170, 10, size=1000)
3
4 def z_score(x):
5     # ¿Que va aqui? (sin for)
6     pass
7
8 z = z_score(alturas)
9 # Verificacion: z.mean() ~ 0, z.std() ~ 1
```

Ejercicio 2: distancias con broadcasting

Distancia euclidiana de cada punto a un centro, sin loops.

```
1 puntos = rng.normal(size=(500, 2))    # 500 puntos en 2D
2 centro = np.array([1.0, 1.0])
3
4 # ¿Que va aqui?
5 # Pista: (puntos - centro) usa broadcasting (500,2)-(2,)
6 # Luego: elevar al cuadrado, sumar con axis=1, raíz
7 distancias = ...
8
9 # ¿Cuántos puntos estan a menos de 1 del centro?
10 cercanos = ...
```

Esta operación exacta es el corazón de k-NN y k-Means (semanas 6 y 11).

Ejercicio 3: análisis de notas

```
1 # notas de 200 estudiantes en 3 parciales
2 notas = rng.integers(40, 101, size=(200, 3))
3
4 # a) promedio de cada estudiante (200,)
5 # b) promedio de cada parcial (3,)
6 # c) ¿cuantos estudiantes aprobaron ( $\geq 61$ ) el curso?
7 # d) ¿que estudiante tuvo el mejor promedio?
```

Resolvemos los tres ejercicios en el notebook de la sesión; lo que falte queda de práctica para la casa.

Lo esencial de hoy

- Un ndarray es un **bloque contiguo** de un solo dtype
- **Vectorizar**: las ufuncs corren en C; el for numérico es sospechoso
- **Máscaras booleanas** para filtrar; ojo con vistas vs copias
- **Broadcasting**: alinear desde la derecha, el 1 se estira
- `axis=k` colapsa el eje k; `%timeit` para medir

Próxima clase (martes 28): Pandas y SQL

Se asigna la **HDT 1** (entrega: martes 4 de agosto)